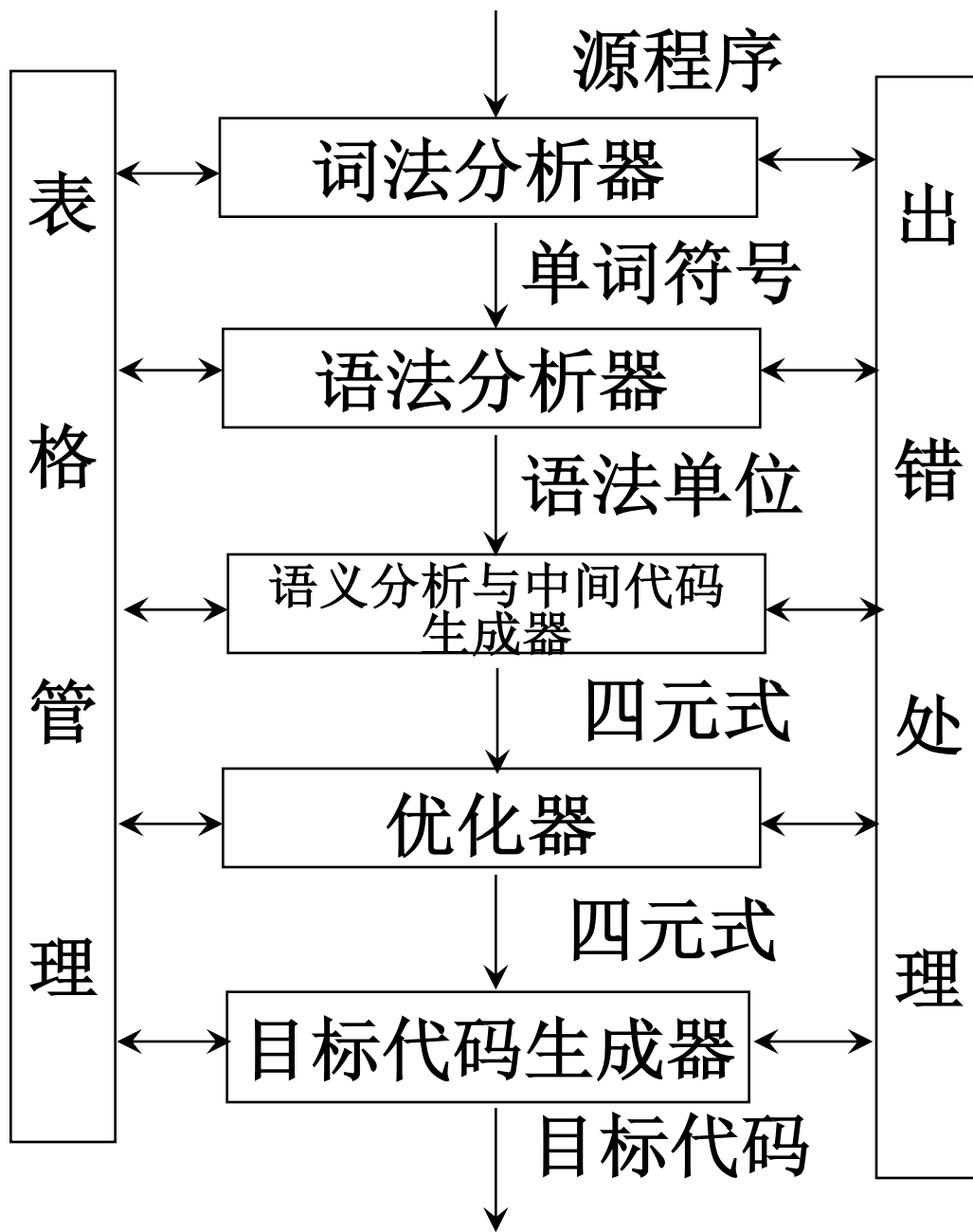


# 第六章

## 属性文法和语法制导翻译



# 第六章

## 属性文法和语法制导翻译

- 6.1 属性文法
- 6.2 基于属性文法的处理方法
- 6.3 S-属性文法的自下而上计算
- 6.4 L-属性文法和自顶向下翻译
- 6.5 自下而上计算继承属性

## 6.1 属性文法

- **属性文法**（也称属性翻译文法）
  - Knuth在1968年提出
  - 在上下文无关文法的基础上，为每个文法符号（终结符或非终结符）配备若干相关的“值”（称为属性）。
  - 属性代表与文法符号相关信息，如类型、值、代码序列、符号表内容等
  - 属性可以进行计算和传递
  - 属性加工的过程即语义处理的过程
  - **语义规则**：对于文法的每个产生式都配备了一组属性的计算规则



Knuth, D. E. : Turing Award, 1974

- 属性
  - 综合属性：“自下而上”传递信息
  - 继承属性：“自上而下”传递信息
- 终结符只有综合属性，由词法分析器提供
- 非终结符既可有综合属性也可有继承属性

- 在一个属性文法中，对应于每个产生式 $A \rightarrow \alpha$ 都有一套与之相关联的语义规则，每条规则的形式为：

$$b := f(c_1, c_2, \dots, c_k)$$

这里， $f$ 是一个函数，而且或者

1.  $b$ 是 **$A$ 的一个综合属性**并且 $c_1, c_2, \dots, c_k$ 是产生式右边文法符号的属性，或者
2.  $b$ 是产生式**右边某个文法符号的一个继承属性**并且 $c_1, c_2, \dots, c_k$ 是 $A$ 或产生式右边任何文法符号的属性。

说明：

- 在两种情况下，属性 $b$ 都依赖于属性 $c_1, c_2, \dots, c_k$
- 属性计算规则中只能使用相应产生式中的文法符号的属性

表6.1 一个简单台式计算器的属性文法

产生式

$L \rightarrow En$

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

语义规则

$\text{print}(E.\text{val})$

$E.\text{val} := E_1.\text{val} + T.\text{val}$

$E.\text{val} := T.\text{val}$

$T.\text{val} := T_1.\text{val} * F.\text{val}$

$T.\text{val} := F.\text{val}$

$F.\text{val} := E.\text{val}$

$F.\text{val} := \text{digit}.\text{lexval}$

- 综合属性

- 在语法树中，一个结点的综合属性的值由其子结点的属性值确定。
- 使用自底向上的方法在每一个结点处使用语法规则计算综合属性的值



表6.1 一个简单台式计算器的属性文法

产生式

$L \rightarrow En$

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

语义规则

$\text{print}(E.\text{val})$

$E.\text{val} := E_1.\text{val} + T.\text{val}$

$E.\text{val} := T.\text{val}$

$T.\text{val} := T_1.\text{val} * F.\text{val}$

$T.\text{val} := F.\text{val}$

$F.\text{val} := E.\text{val}$

$F.\text{val} := \text{digit}.\text{lexval}$

## 产生式

## 语义规则

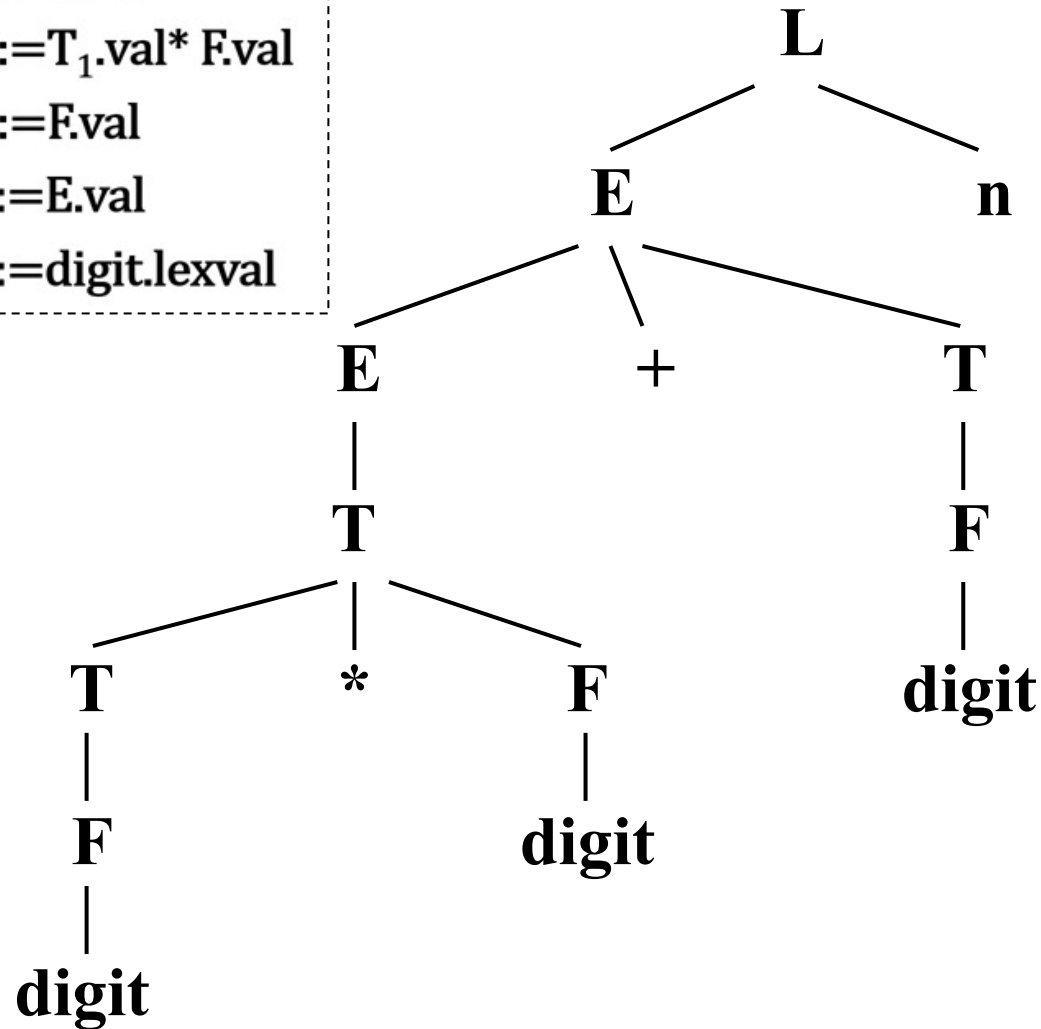
$$L \rightarrow \text{En}$$

```
print(E.val)
```

$$E \rightarrow E_1 + T$$
$$E.val := E_1.val + T.val$$
$$E \rightarrow T$$
$$E.val := T.val$$
$$T \rightarrow T_1^* F$$
$$T.val := T_1.val * F.val$$
$$T \rightarrow F$$
$$T.val := F.val$$
$$F \rightarrow (E)$$
$$F.val := E.val$$
$$F \rightarrow \text{digit}$$

**F.val := digit.lexval**

## 3\*5+4n的帶注释的语法树



产生式

$L \rightarrow En$

$E \rightarrow E_1 + T$

$E \rightarrow T$

$T \rightarrow T_1 * F$

$T \rightarrow F$

$F \rightarrow (E)$

$F \rightarrow \text{digit}$

语义规则

$\text{print}(E.\text{val})$

$E.\text{val} := E_1.\text{val} + T.\text{val}$

$E.\text{val} := T.\text{val}$

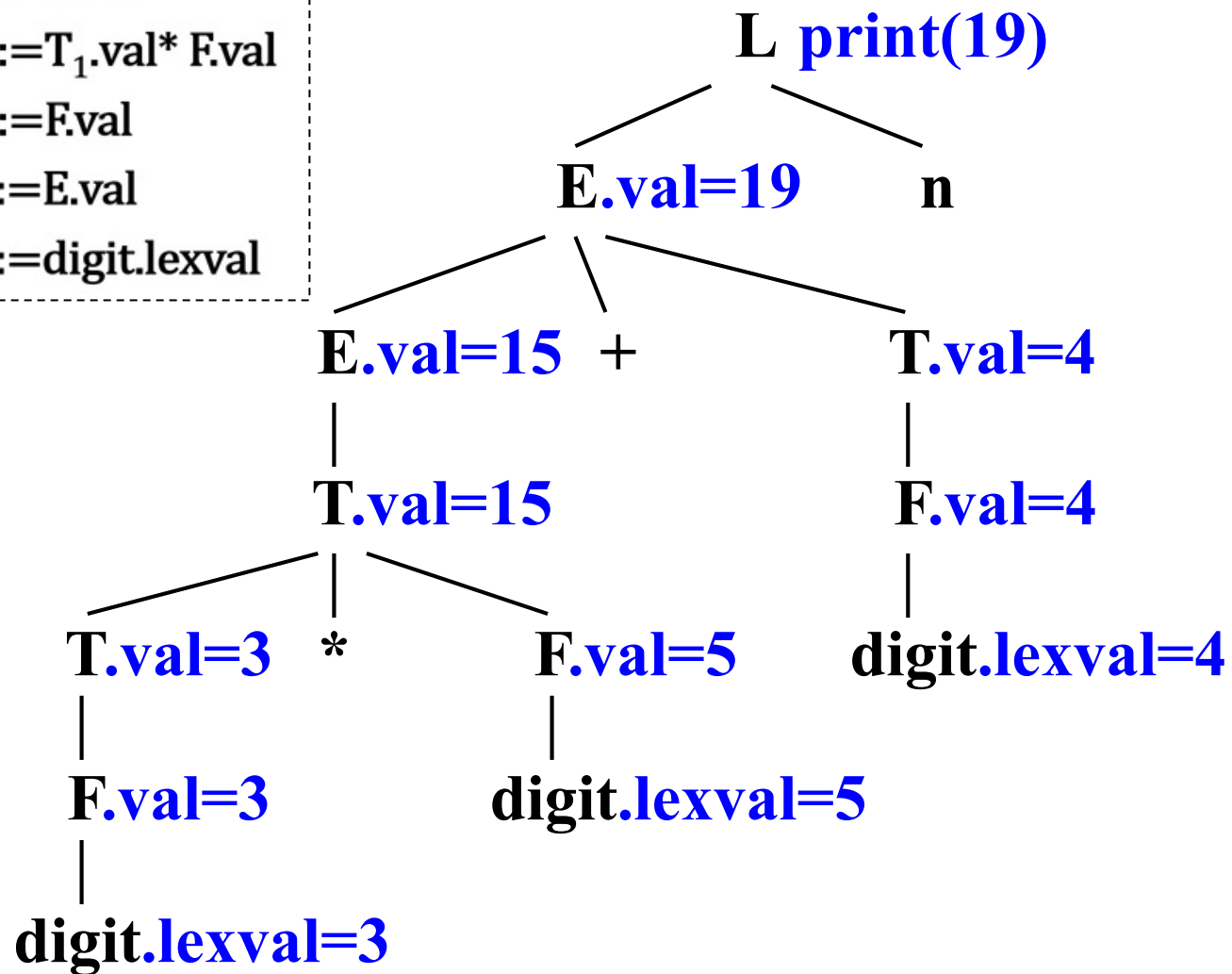
$T.\text{val} := T_1.\text{val} * F.\text{val}$

$T.\text{val} := F.\text{val}$

$F.\text{val} := E.\text{val}$

$F.\text{val} := \text{digit}.\text{lexval}$

3\*5+4n的带注释的语法树



- 继承属性

- 在语法树中，一个结点的继承属性由此结点的父结点和/或兄弟结点的某些属性确定
- 用继承属性来表示程序设计语言结构中的上下文依赖关系很方便

表6.2 带继承属性L.in的属性文法

产生式

$$D \rightarrow TL$$

$$T \rightarrow \text{int}$$

$$T \rightarrow \text{real}$$

$$L \rightarrow L_1, \text{id}$$

$$L \rightarrow \text{id}$$

表6.2 带继承属性L.in的属性文法

产生式

$D \rightarrow TL$

$T \rightarrow \text{int}$

$T \rightarrow \text{real}$

$L \rightarrow L_1, \text{id}$

$L \rightarrow \text{id}$

语义规则

$L.in := T.type$

$T.type := \text{integer}$

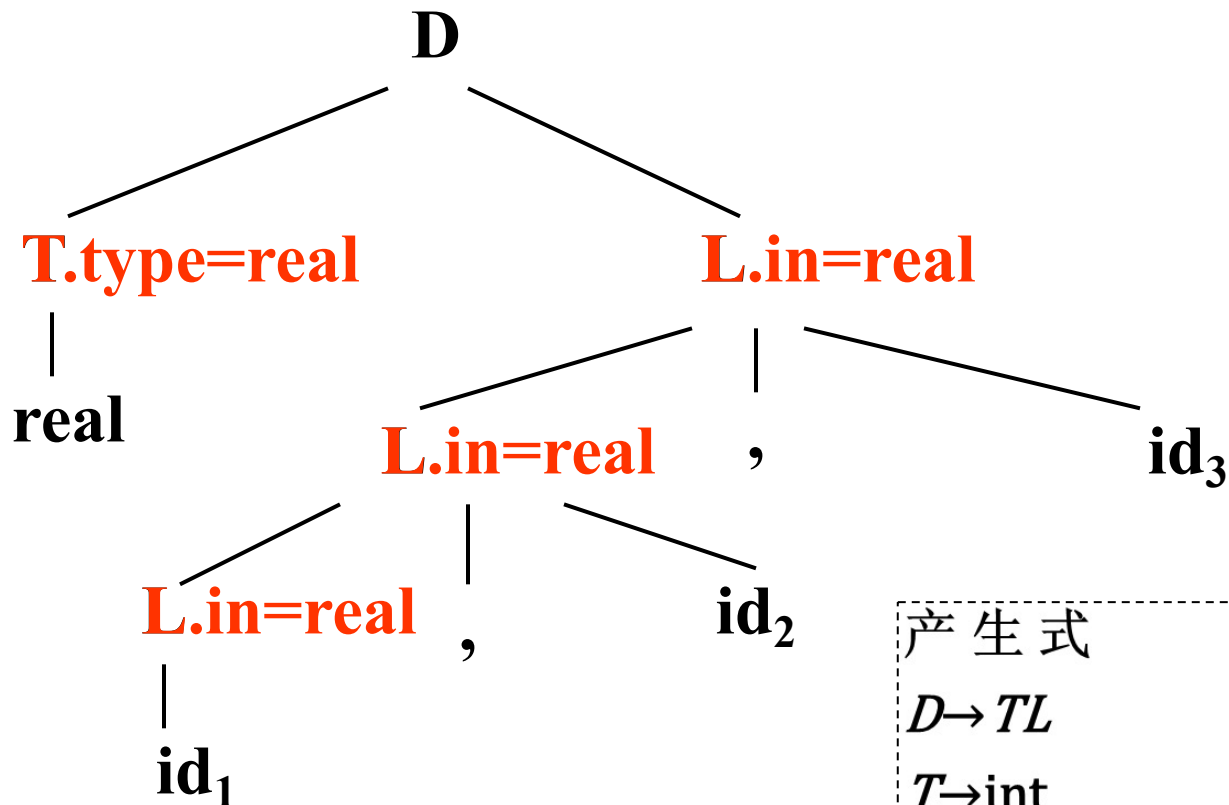
$T.type := \text{real}$

$L_1.in := L.in$

$\text{addtype}(\text{id.entry}, L.in)$

$\text{addtype}(\text{id.entry}, L.in)$

# 句子 $\text{real id}_1, \text{id}_2, \text{id}_3$ 的带注释的语法树



| Name          | type |
|---------------|------|
| ...           | ...  |
| $\text{id}_1$ | real |
| $\text{id}_2$ | real |
| $\text{id}_3$ | real |

产生式

$D \rightarrow TL$

$T \rightarrow \text{int}$

$T \rightarrow \text{real}$

$L \rightarrow L_1, \text{id}$

$L \rightarrow \text{id}$

语义规则

$L.\text{in} := T.\text{type}$

$T.\text{type} := \text{integer}$

$T.\text{type} := \text{real}$

$L_1.\text{in} := L.\text{in}$

$\text{addtype}(\text{id.entry}, L.\text{in})$

$\text{addtype}(\text{id.entry}, L.\text{in})$

## 6.2 基于属性文法的的处理方法

输入串  $\longrightarrow$  语法树  $\longrightarrow$  依赖图  $\longrightarrow$  语义规则计算次序

- 由源程序的语法结构所驱动的处理办法就是**语法制导翻译法**。
- 语义规则的计算
  - 产生代码
  - 在符号表中存放信息
  - 给出错误信息
  - 执行任何其它动作
- 对输入符号串的**翻译**也就是根据语义规则进行**计算**的结果。



## 6.2 基于属性文法的的处理方法

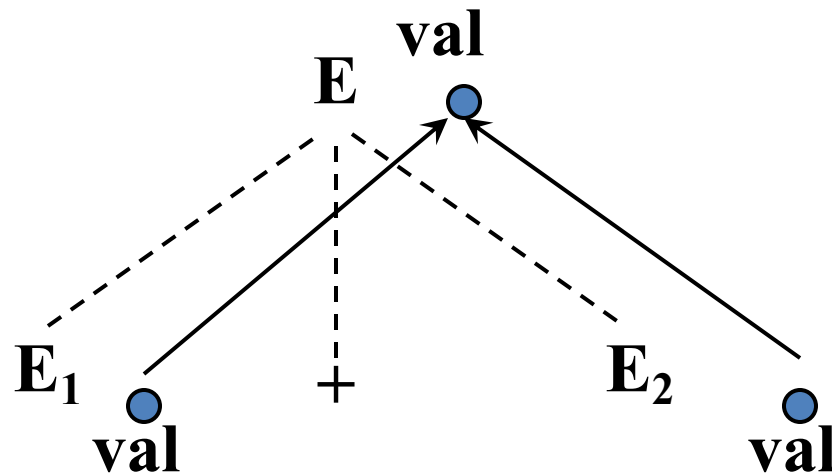
- 6.2.1 依赖图
- 6.2.2 树遍历的属性计算方法
- 6.2.3 一遍扫描的处理方法
- 6.2.4 抽象语法树

## 6.2.1 依赖图

- 在一棵语法树中的结点的继承属性和综合属性之间的相互依赖关系可以由称作**依赖图**的一个**有向图**来描述。
- 依赖图中为每一个属性设置一个结点，**如果属性b依赖于属性c，则从属性c的结点有一条有向边连到属性b的结点。**

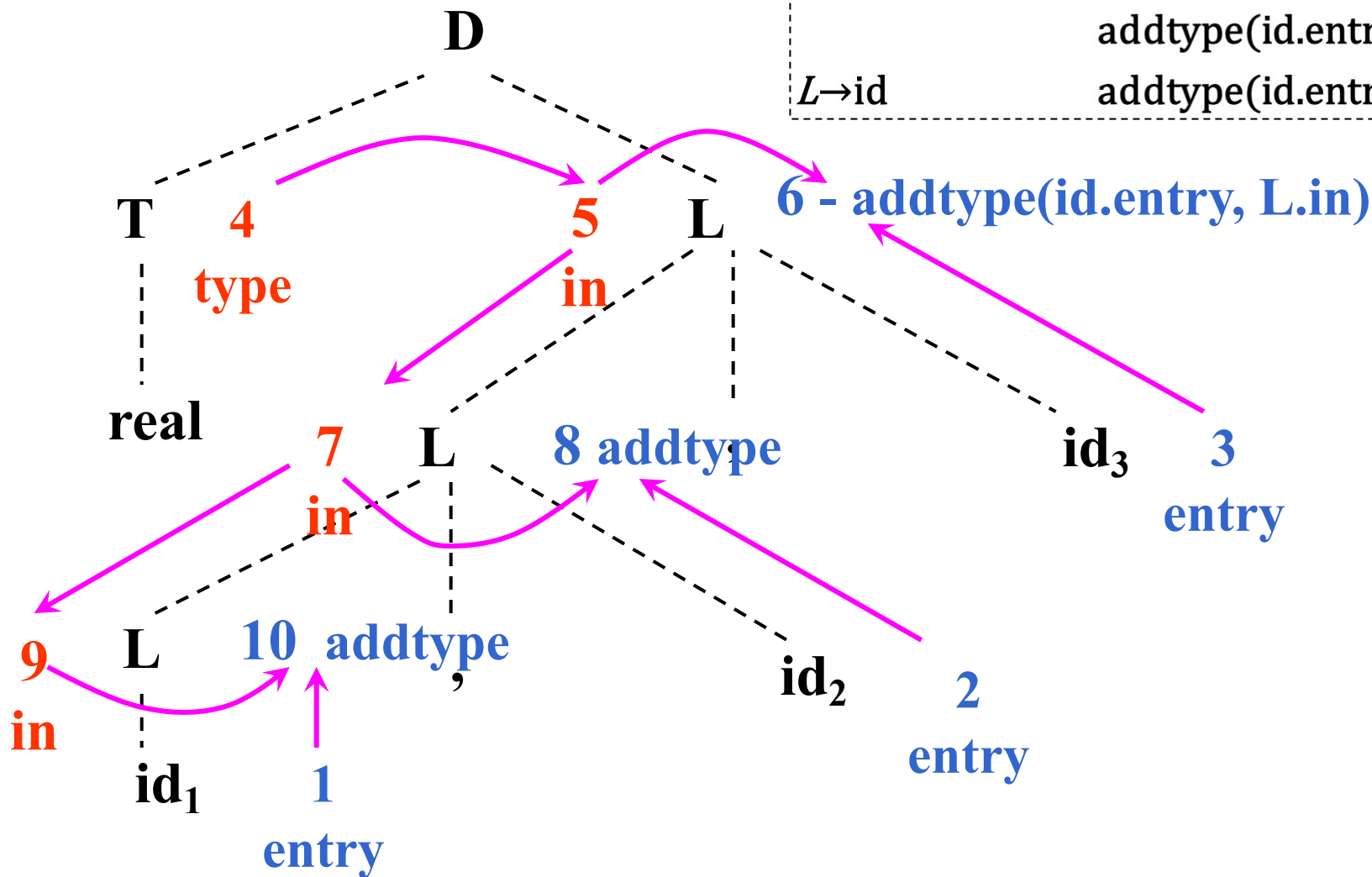
$E \rightarrow E_1 + E_2$

$E.val := E_1.val + E_2.val$



# 句子 $\text{real id}_1, \text{id}_2, \text{id}_3$ 的带注释的语法树的依赖图

| 产生式                         | 语义规则                             |
|-----------------------------|----------------------------------|
| $D \rightarrow TL$          | $L.in := T.type$                 |
| $T \rightarrow \text{int}$  | $T.type := \text{integer}$       |
| $T \rightarrow \text{real}$ | $T.type := \text{real}$          |
| $L \rightarrow L_1, id$     | $L_1.in := L.in$                 |
| $L \rightarrow id$          | $\text{addtype}(id.entry, L.in)$ |



- 如果一属性文法不存在属性之间的循环依赖关系，那么称该文法为良定义的。

如  $p$ 、 $c_1$ 、 $c_2$  都是属性，

求值规则：  $p := f_1(c_1)$

$c_1 := f_2(c_2)$

$c_2 := f_3(p)$

$p$ 、 $c_1$ 、 $c_2$  循环依赖。

## 6.2.1 依赖图

- 属性的计算次序

- 一个有向非循环图的**拓扑序**是图中结点的任何顺序  $m_1, m_2, \dots, m_k$ , 使得边必须是从序列中前面的结点指向后面的结点。

如果  $m_i \rightarrow m_j$  是  $m_i$  到  $m_j$  的一条边, 则在序列中  $m_i$  必须出现在  $m_j$  之前。

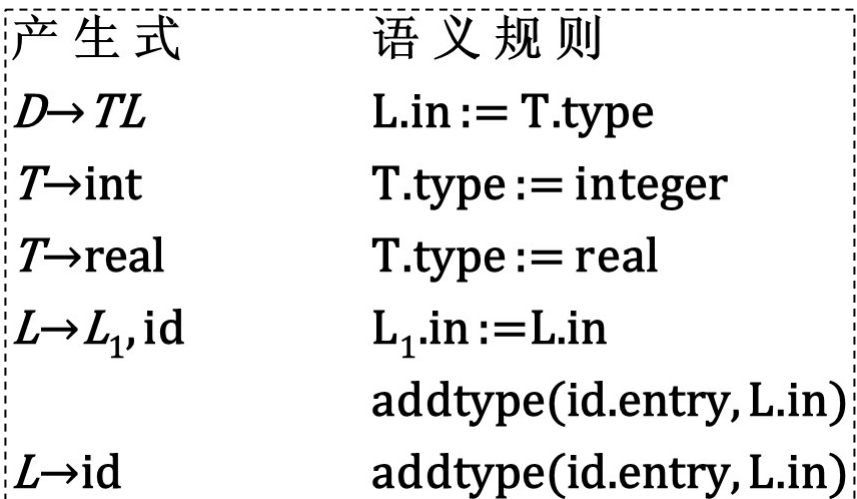
- 一个依赖图的任何拓扑排序都给出一个语法树中结点的语义规则计算的有效顺序。

# 属性的计算次序

- 属性文法说明的翻译是很精确的
  - 基础文法用于建立输入符号串的语法分析树
  - 根据语义规则建立依赖图
  - 从依赖图的拓扑排序中，可以得到计算语义规则的顺序
  - 用这个顺序计算语义规则就得到输入符号串的翻译

输入串  $\longrightarrow$  语法树  $\longrightarrow$  依赖图  $\longrightarrow$  语义规则计算次序

## 拓扑排序： 低序号到高序号



```

a4 := real;
a5 := a4
addtype (id3.entry, a5);
a7 := a5;
addtype (id2.entry, a7);
a9 := a7
addtype (id1.entry, a9);

```

24



## 6.2.2 树遍历的属性计算方法

- 通过树遍历的方法计算属性的值
  - 假设语法树已建立，且树中已带有开始符号的继承属性和终结符的综合属性
  - 以某种次序遍历语法树，直至计算出所有属性
    - 深度优先，从左到右的遍历

While 还有未被计算的属性 do  
    VisitNode(S)     /\*S是开始符号\*/

```
procedure VisitNode (N:Node) ;  
begin  
    if N是一个非终结符 then  
        /*假设它的产生式为 $N \rightarrow X_1 \dots X_m$ */  
        for i := 1 to m do  
            if  $X_i \in V_N$  then /*即 $X_i$ 是非终结符*/  
                begin  
                    计算 $X_i$ 的所有能够计算的继承属性;  
                    VisitNode ( $X_i$ )  
                end;  
                计算N的所有能够计算的综合属性;  
        end  
end
```

## 6.2.2 树遍历的属性计算方法

### 例6.6

产生式

$S \rightarrow XYZ$

$X \rightarrow x$

$Y \rightarrow y$

$Z \rightarrow z$

语义规则

$X.i := Z.s$

$Y.i := S.s$

$Z.i := S.i$

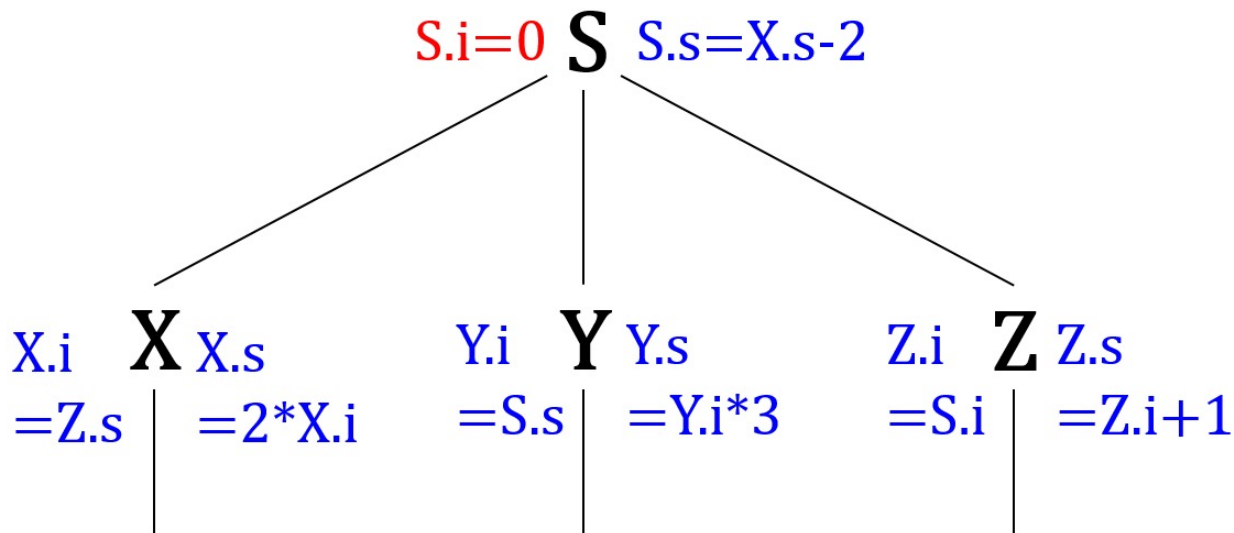
$S.s := X.s - 2$

$X.s := 2 * X.i$

$Y.s := Y.i * 3$

$Z.s := Z.i + 1$

设  $S.i=0$ ,  
输入串: **xyz**



第1遍扫描:

$S.i=0$   
 $X.i=Z.s$   
 $X.s=2*X.i$   
 $Y.i=S.s$   
 $Y.s=Y.i*3$   
 $Z.i=S.i=0$   
 $Z.s=Z.i+1=1$   
 $S.s=X.s-2$

第2遍扫描:

$S.i=0$   
 $X.i=Z.s=1$   
 $X.s=2*X.i=2$   
 $Y.i=S.s$   
 $Y.s=Y.i*3$   
 $Z.i=S.i=0$   
 $Z.s=Z.i+1=1$   
 $S.s=X.s-2=0$

第3遍扫描:

$S.i=0$   
 $X.i=Z.s=1$   
 $X.s=2*X.i=2$   
 $Y.i=S.s=0$   
 $Y.s=Y.i*3=0$   
 $Z.i=S.i=0$   
 $Z.s=Z.i+1=1$   
 $S.s=X.s-2=0$

## 6.2.3 一遍扫描的处理方法

- 一遍扫描的处理方法是在语法分析的同时计算属性值
  - 所采用的语法分析方法
  - 属性的计算次序
- L-属性文法：一遍扫描的自上而下分析
- S-属性文法：一遍扫描的自下而上分析

## 6.2.3 一遍扫描的处理方法

- 一遍扫描的语法制导翻译法

为文法中每个产生式配上一组语义规则，并且在语法分析的同时执行这些语义规则。

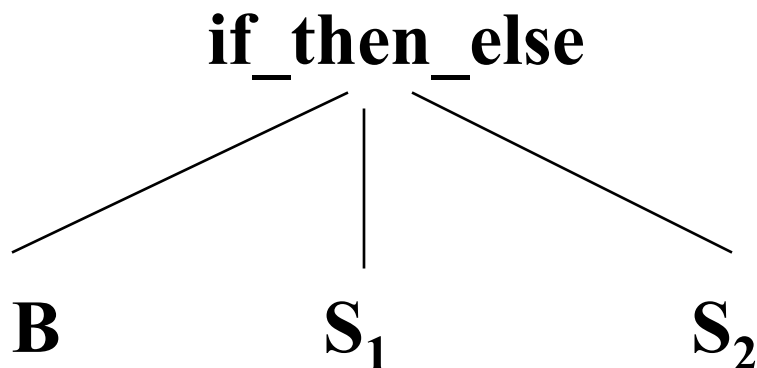
- 语义规则被计算的时机

- 在自上而下语法分析中，一个产生式匹配输入串成功时
- 在自下而上分析中，当一个产生式被用于进行归约时

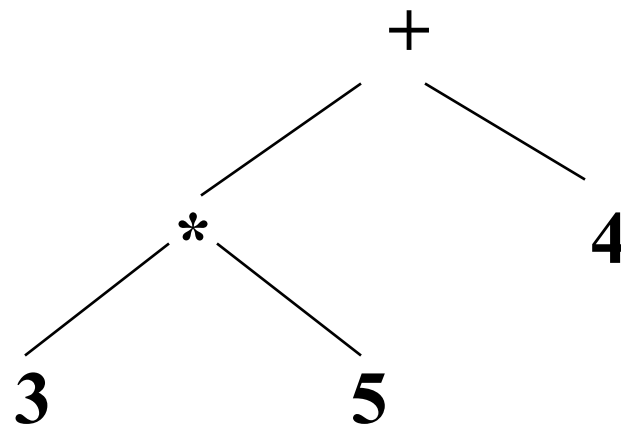
## 6.2.4 抽象语法树

- 在语法树中去掉那些对翻译不必要的信息，从而获得更有效的源程序中间表示。这种经变换后的语法树称之为**抽象语法树**(Abstract Syntax Tree)
- 在抽象语法树中，操作符和关键字不作为叶结点出现，而是作为内部结点。

□  $S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$



□  $3 * 5 + 4$



## 6.2.4 抽象语法树

- 建立抽象语法树中的结点：
  - 运算符结点： `mknnode (op, left, right)`  
标号是`op`，两个域`left`和`right`分别指向左子树和右子树。
  - 标识符结点： `mkleaf (id, entry)`  
标号为`id`，一个域`entry`指向标识符在符号表中的入口。
  - 数结点： `mkleaf (num, val)`  
标号为`num`，一个域`val`用于存放数的值。



# 建立抽象语法树的语义规则

产生式

语义规则

$E \rightarrow E_1 + T$        $E.nptr := \text{mknode}('+', E_1.nptr, T.nptr)$

$E \rightarrow E_1 - T$        $E.nptr := \text{mknode}('-', E_1.nptr, T.nptr)$

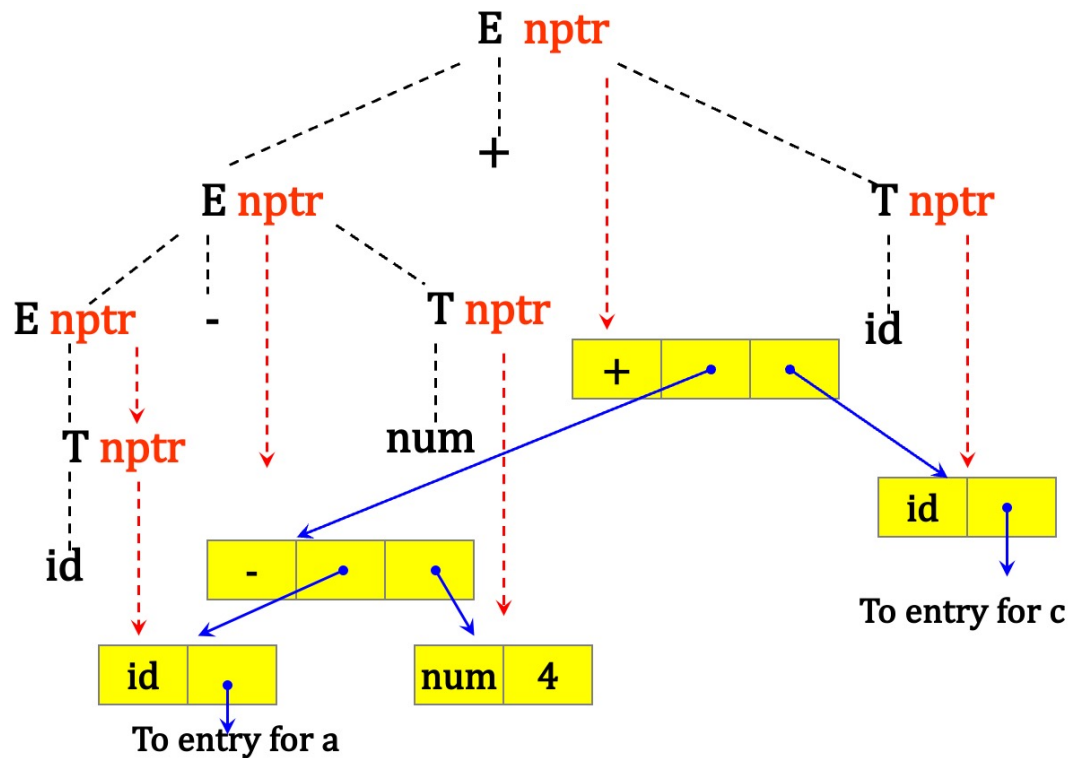
$E \rightarrow T$        $E.nptr := T.nptr$

$T \rightarrow (E)$        $T.nptr := E.nptr$

$T \rightarrow \text{id}$        $T.nptr := \text{mkleaf}(\text{id}, \text{id.entry})$

$T \rightarrow \text{num}$        $T.nptr := \text{mkleaf}(\text{num}, \text{num.val})$

# a-4+c的抽象语法树（自下而上分析）

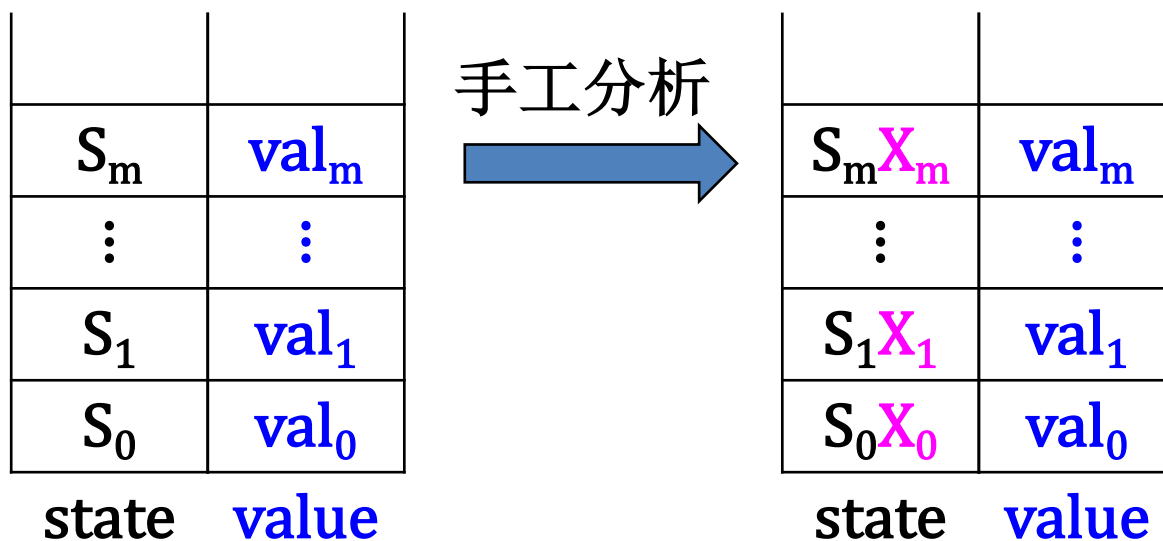


## 6.3 S-属性文法的自下而上计算

- **S-属性文法**：只含有综合属性
- 综合属性可以在分析输入符号串的同时由自下而上的分析器来计算。
- 分析器可以**保存**与栈中文法符号有关的综合属性值，每当进行归约时，新的属性值就由栈中正在归约的产生式右边符号的属性值来**计算**。

## 6.3 S-属性文法的自下而上计算

- S-属性文法的翻译器借助于LR分析器实现。  
文法符号隐含在state中不需存储在栈中。  
在分析栈中使用一个附加的域来存放综合属性值。



## 6.3 S-属性文法的自下而上计算

- 综合属性在每次归约前计算。

产生式:  $E \rightarrow E_1 + T$

语义规则:  $E.val := E_1.val + T.val$

归约前

|       |                |       |
|-------|----------------|-------|
|       |                |       |
|       |                |       |
| top → | T              | 5     |
|       | +              | -     |
|       | E <sub>1</sub> | 3     |
|       | state          | value |

归约后

|       |       |       |
|-------|-------|-------|
|       |       |       |
|       |       |       |
|       | T     | 5     |
|       | +     | -     |
| top → | E     | 8     |
|       | state | value |

$val[top-2] = val[top-2] + val[top], top = top - 2$

## 6.3 S-属性文法的自下而上计算

表6.5 用LR分析器实现台式计算器

| 产生式                          | 语义规则                                            | 代码段                                                                                                             |
|------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| $L \rightarrow En$           | $\text{print}(E.\text{val})$                    | $\text{print}(\text{val}[\text{top}-1])$<br>$\text{top} := \text{top}-1$                                        |
| $E \rightarrow E_1 + T$      | $E.\text{val} := E_1.\text{val} + T.\text{val}$ | $\text{val}[\text{top}-2] := \text{val}[\text{top}-2] + \text{val}[\text{top}]$<br>$\text{top} := \text{top}-2$ |
| $E \rightarrow T$            | $E.\text{val} := T.\text{val}$                  |                                                                                                                 |
| $T \rightarrow T_1 * F$      | $T.\text{val} := T_1.\text{val} * F.\text{val}$ | $\text{val}[\text{top}-2] := \text{val}[\text{top}-2] * \text{val}[\text{top}]$<br>$\text{top} := \text{top}-2$ |
| $T \rightarrow F$            | $T.\text{val} := F.\text{val}$                  |                                                                                                                 |
| $F \rightarrow (E)$          | $F.\text{val} := E.\text{val}$                  | $\text{val}[\text{top}-2] := \text{val}[\text{top}-1]$<br>$\text{top} := \text{top}-2$                          |
| $F \rightarrow \text{digit}$ | $F.\text{val} := \text{digit.lexval}$           |                                                                                                                 |

## 6.3 S-属性文法的自下而上计算

- 台式计算器拓广文法  $G(L')$

(7)  $L' \rightarrow L$

(0)  $L \rightarrow En$

(1)  $E \rightarrow E + T$

(2)  $E \rightarrow T$

(3)  $T \rightarrow T * F$

(4)  $T \rightarrow F$

(5)  $F \rightarrow (E)$

(6)  $F \rightarrow i$

| 状态 | ACTION |    |    |    |     |     |     | GOTO |   |   |    |
|----|--------|----|----|----|-----|-----|-----|------|---|---|----|
|    | d      | +  | *  | (  | )   | n   | #   | L    | E | T | F  |
| 0  | s5     |    |    | s4 |     |     |     | 12   | 1 | 2 | 3  |
| 1  |        | s6 |    |    |     | s13 |     |      |   |   |    |
| 2  |        | r2 | s7 |    | r2  | r2  |     |      |   |   |    |
| 3  |        | r4 | r4 |    | r4  | r4  |     |      |   |   |    |
| 4  | s5     |    |    | s4 |     |     |     |      | 8 | 2 | 3  |
| 5  |        | r6 | r6 |    | r6  | r6  |     |      |   |   |    |
| 6  | s5     |    |    | s4 |     |     |     |      |   | 9 | 3  |
| 7  | s5     |    |    | s4 |     |     |     |      |   |   | 10 |
| 8  |        | s6 |    |    | s11 |     |     |      |   |   |    |
| 9  |        | r1 | s7 |    | r1  | r1  |     |      |   |   |    |
| 10 |        | r3 | r3 |    | r3  | r3  |     |      |   |   |    |
| 11 |        | r5 | r5 |    | r5  | r5  |     |      |   |   |    |
| 12 |        |    |    |    |     |     | acc |      |   |   |    |
| 13 |        |    |    |    |     |     | r0  |      |   |   |    |

台式计算器文法SLR(1)分析表

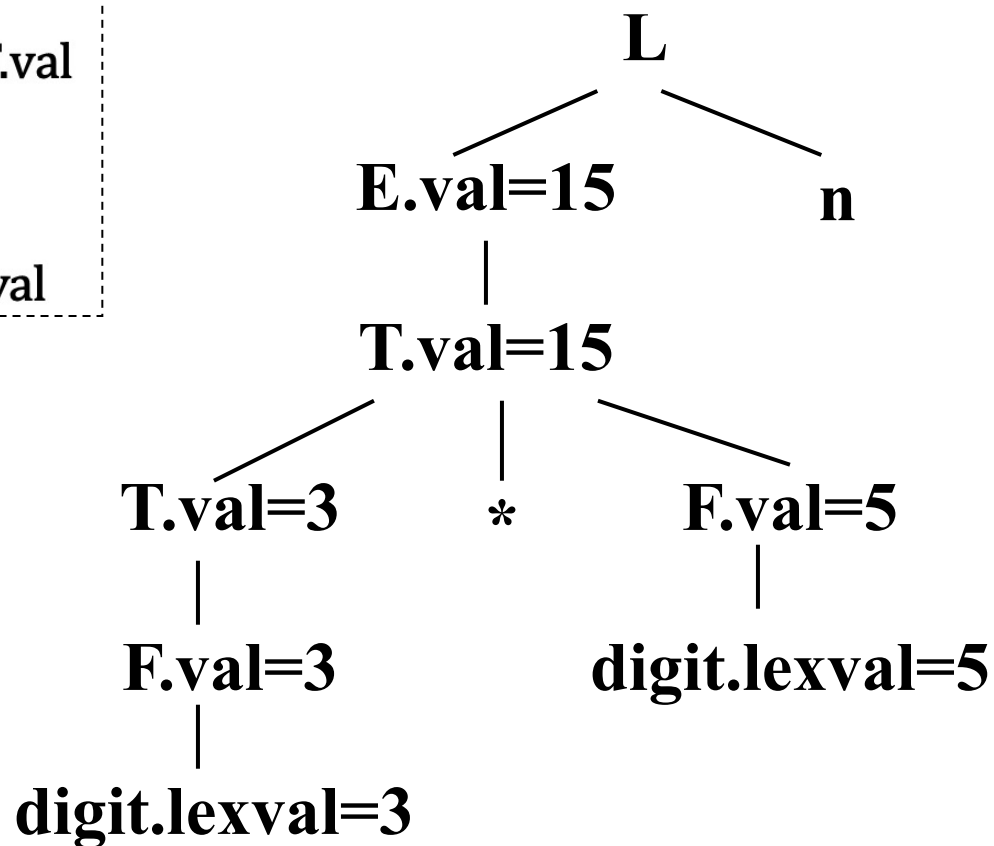


输入:  $3*5n$

| state         |       | value | 输入       | 动作                       |
|---------------|-------|-------|----------|--------------------------|
| 0             | #     | -     | $3*5n\#$ |                          |
| 05            | #3    | -3    | $*5n\#$  | s5                       |
| 03            | # F   | -3    | $*5n\#$  | r6:F $\rightarrow$ digit |
| 02            | # T   | -3    | $*5n\#$  | r4:T $\rightarrow$ F     |
| 027           | # T*  | -3-   | $5n\#$   | s7                       |
| 0275          | # T*5 | -3-5  | $n\#$    | s5                       |
| 027 <u>10</u> | # T*F | -3-5  | $n\#$    | r6:F $\rightarrow$ digit |
| 02            | # T   | -15   | $n\#$    | r3:T $\rightarrow$ T*F   |
| 01            | # E   | -15   | $n\#$    | r2:E $\rightarrow$ T     |
| 01 <u>13</u>  | # En  | -15-  | #        | s13                      |
| 01 <u>2</u>   | # L   | -15   | #        | r0:L $\rightarrow$ En    |
|               |       |       |          | acc                      |

| 产生式                          | 语义规则                                 |
|------------------------------|--------------------------------------|
| $L \rightarrow En$           | <code>print(E.val)</code>            |
| $E \rightarrow E_1 + T$      | <code>E.val := E1.val + T.val</code> |
| $E \rightarrow T$            | <code>E.val := T.val</code>          |
| $T \rightarrow T_1 * F$      | <code>T.val := T1.val * F.val</code> |
| $T \rightarrow F$            | <code>T.val := F.val</code>          |
| $F \rightarrow (E)$          | <code>F.val := E.val</code>          |
| $F \rightarrow \text{digit}$ | <code>F.val := digit.lexval</code>   |

## 3\*5的带注释的语法树



## 6.4 L-属性文法和自顶向下翻译

- 一个属性文法称为**L-属性文法**，如果对于每个产生式 $A \rightarrow X_1 X_2 \dots X_n$ ，其每个语义规则中的每个属性或者是综合属性，或者是 $X_j (1 \leq j \leq n)$ 的一个继承属性且这个继承属性仅依赖于：
  - (1) 产生式 $X_j$ 的左边符号 $X_1, X_2, \dots, X_{j-1}$ 的属性；
  - (2)  $A$ 的继承属性。
- S-属性文法一定是L-属性文法

判断下列文法是否为L-属性文法:

产生式

$A \rightarrow LM$

$A \rightarrow QR$

语义规则

$L.i := l(A.i)$

$M.i := m(L.s)$

$R.i := r(A.i)$

$Q.i := q(R.s)$  ✗

$A.s := f(Q.s)$

## 6.4.1 翻译模式

- 属性文法：语言翻译的规范说明，隐去实现细节。
- **翻译模式**：给出了使用语义规则进行计算的次序，这样就可把某些实现细节表示出来。
- 在翻译模式中，和文法符号相关的属性和语义规则（这里我们也称**语义动作**），用花括号**{ }**括起来，插入到产生式右部的合适位置上。

## 6.4.1 翻译模式

例：将中缀表达式翻译成相应的后缀表达式。

如：9-5+2 翻译为：95-2+。

$E \rightarrow TR$

$R \rightarrow \text{addop } T \{ \text{print}(\text{addop.lexeme}) \} R_1 \mid \varepsilon$

$T \rightarrow \text{num} \{ \text{print}(\text{num.val}) \}$

addop表示+或-

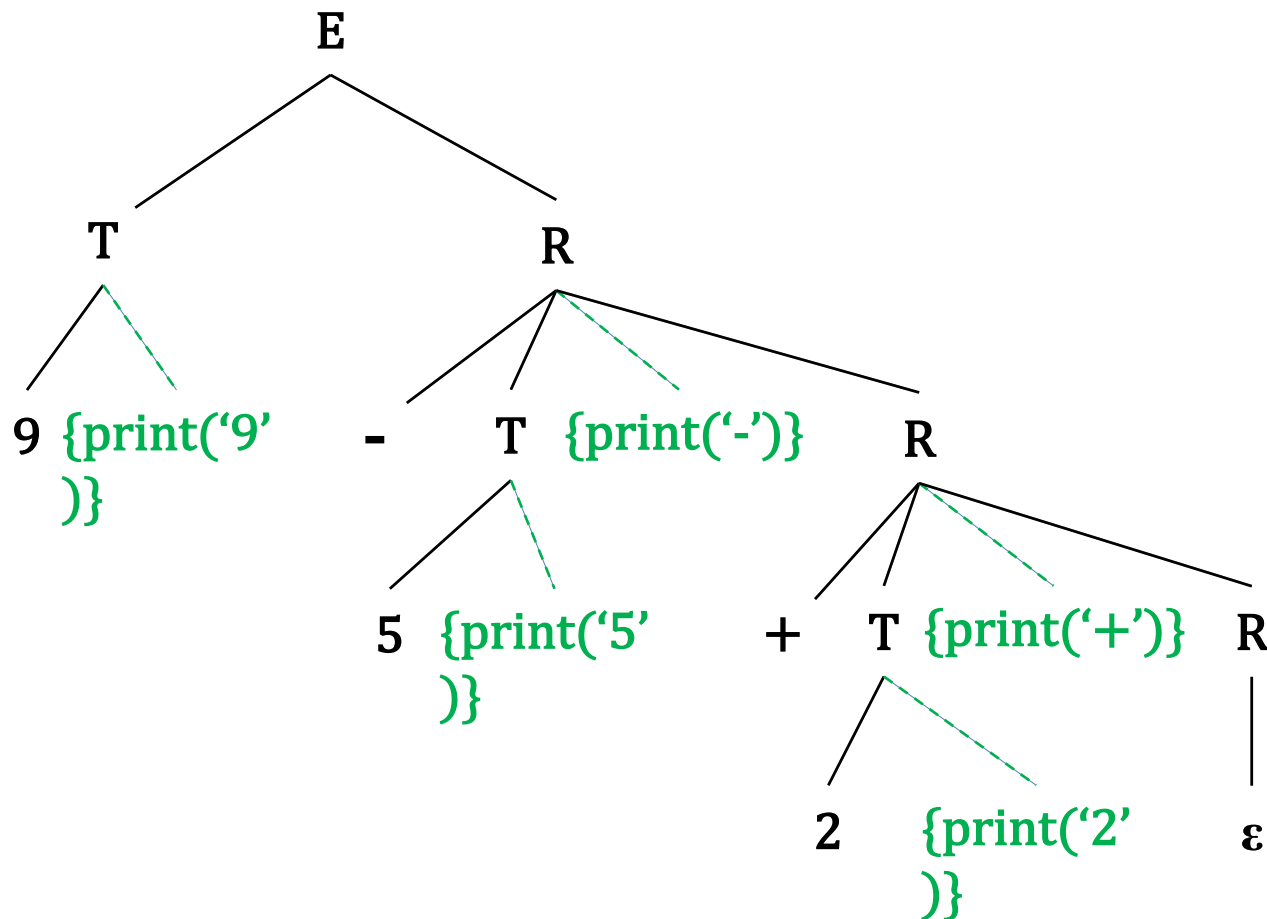
构造语法分析树时，每个语义动作作为相应产生式左部符号结点的儿子。

$E \rightarrow TR$

$R \rightarrow \text{addop } T \{ \text{print}(\text{addop.lexeme}) \} R_1 \mid \epsilon$

$T \rightarrow \text{num} \{ \text{print}(\text{num.val}) \}$

9-5+2



```
void E( )
{ T( ); R( ); }
```

```
void R( )
{ if sym==addop
  { advance( );
    T( );
    printf(aop.lex);
    R( ); }
}
```

```
void T( )
{ if sym==num
  { advance( );
    printf(num.val);
  }
}
```

按深度优先执行语法分析树中的动作，打印输出：95-2+

# 设计翻译模式的原则

- 设计翻译模式时，必须保证当某个动作引用一个属性时它必须是有定义的。
- L-属性文法本身就能确保每个动作不会引用尚未计算出来的属性。



# 建立翻译模式

- 当只需要综合属性时：为每一个语义规则建立一个包含赋值的动作，并把这个动作放在相应的产生式右边的末尾。

产生式

语义规则

$T \rightarrow T_1 * F$

$T.val := T_1.val \times F.val$

建立产生式和语义动作：

$T \rightarrow T_1 * F$

$\{T.val := T_1.val \times F.val\}$

# 建立翻译模式

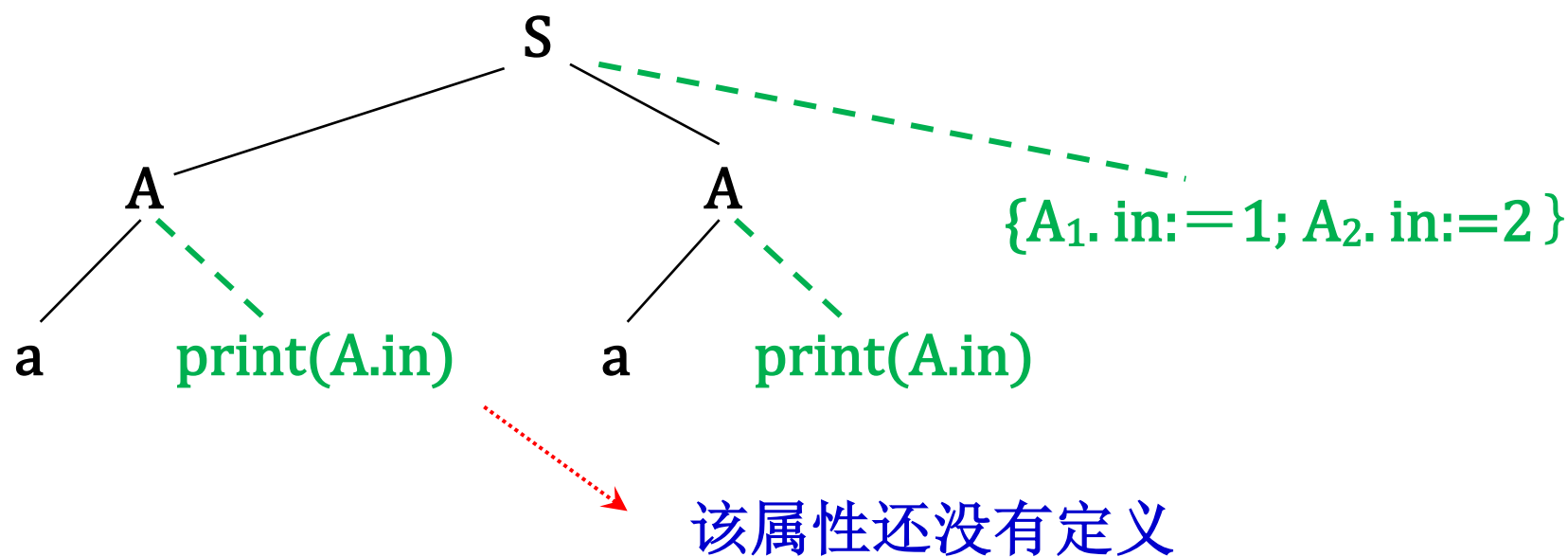
- 如果既有综合属性又有继承属性，在建立翻译模式时必须保证：
  1. 产生式右边的符号的继承属性必须在这个符号以前的动作中计算出来。
  2. 一个动作不能引用这个动作右边的符号的综合属性。
  3. 产生式左边非终结符的综合属性只有在它所引用的所有属性都计算出来以后才能计算。计算这种属性的动作通常可放在产生式右端的末尾。

下面的翻译模式**不满足**上述三个条件中的第一个条件:  
(1)产生式右边的符号的继承属性必须在这个符号**以前**  
**的动作**中计算出来。

$S \rightarrow A_1 A_2 \{ A_1. in := 1; A_2. in := 2 \}$

$A \rightarrow a \{ \text{print}(A. in) \}$

输入串: aa

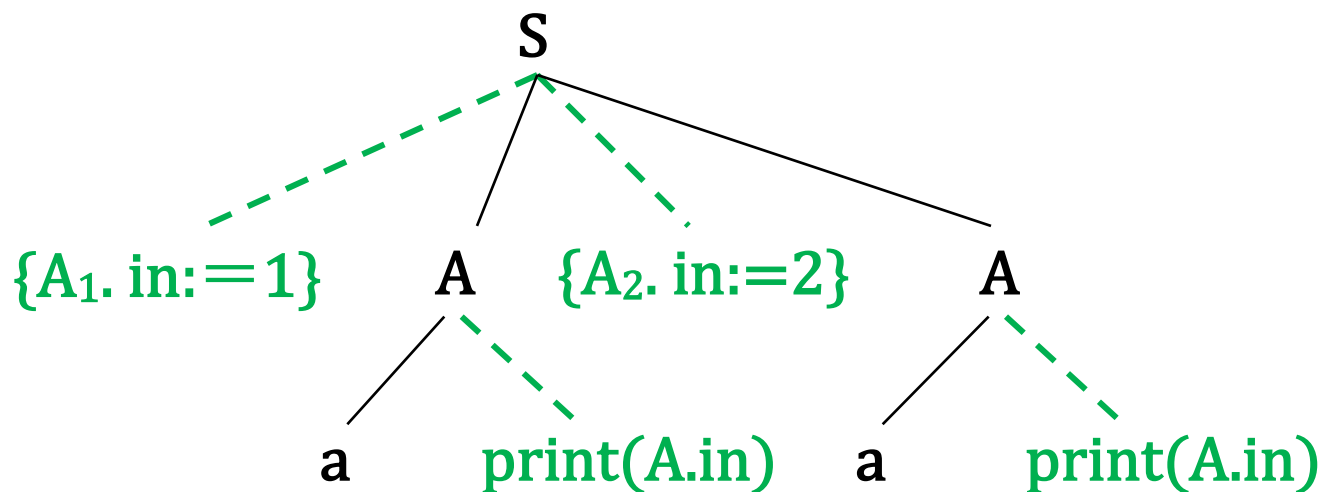


可以改为  $S \rightarrow \{A_1. in := 1\} A_1 \{ A_2. in := 2 \} A_2$

$S \rightarrow \{A_1. \text{in} := 1\} A_1 \{A_2. \text{in} := 2\} A_2$

$A \rightarrow a \{ \text{print}(A. \text{in}) \}$

输入串: aa



## 6.4.2 自顶向下翻译

- L-属性文法在自顶向下翻译中的实现
- 动作是在处于相同位置上的符号被展开（匹配成功）时执行的。
- 为了构造不带回溯的自顶向下语法分析，必须消除文法中的左递归
- 当消除一个翻译模式的基本文法的左递归时同时考虑属性。

- 算术表达式的左递归文法的翻译模式

$$E \rightarrow E_1 + T \quad \{E.val := E_1.val + T.val\}$$
$$E \rightarrow E_1 - T \quad \{E.val := E_1.val - T.val\}$$
$$E \rightarrow T \quad \{E.val := T.val\}$$
$$T \rightarrow (E) \quad \{T.val := E.val\}$$
$$T \rightarrow \text{num} \quad \{T.val := \text{num.val}\}$$

## 左递归

$E \rightarrow E_1 + T$        $\{E.val := E_1.val + T.val\}$

$E \rightarrow E_1 - T$        $\{E.val := E_1.val - T.val\}$

$E \rightarrow T$        $\{E.val := T.val\}$

$T \rightarrow (E)$        $\{T.val := E.val\}$

$T \rightarrow num$        $\{T.val := num.val\}$

## 消除左递归

$E \rightarrow T$        $\{R.i := T.val\}$

$R$        $\{E.val := R.s\}$

$R \rightarrow +$   
     $T$        $\{R_1.i := R.i + T.val\}$

$R_1$        $\{R.s := R_1.s\}$

$R \rightarrow -$   
     $T$        $\{R_1.i := R.i - T.val\}$

$R_1$        $\{R.s := R_1.s\}$

$R \rightarrow \epsilon$        $\{R.s := R.i\}$

$T \rightarrow (E)$        $\{T.val := E.val\}$

$T \rightarrow num$        $\{T.val := num.val\}$

继承属性 $R.i$ : 表示在 $R$ 之前已经推导出的子表达式的值;

综合属性 $R.s$ : 表示在 $R$ 完全展开之后得到的表达式的值.

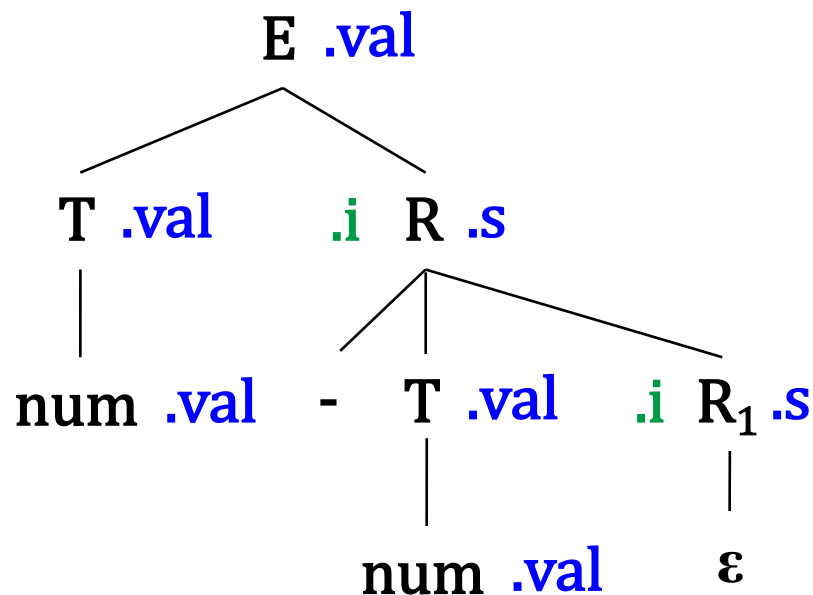


图6-a 表达式 9-5 带注释语法树

## 6.4.2 自顶向下翻译

- L-属性文法在自顶向下翻译中的实现

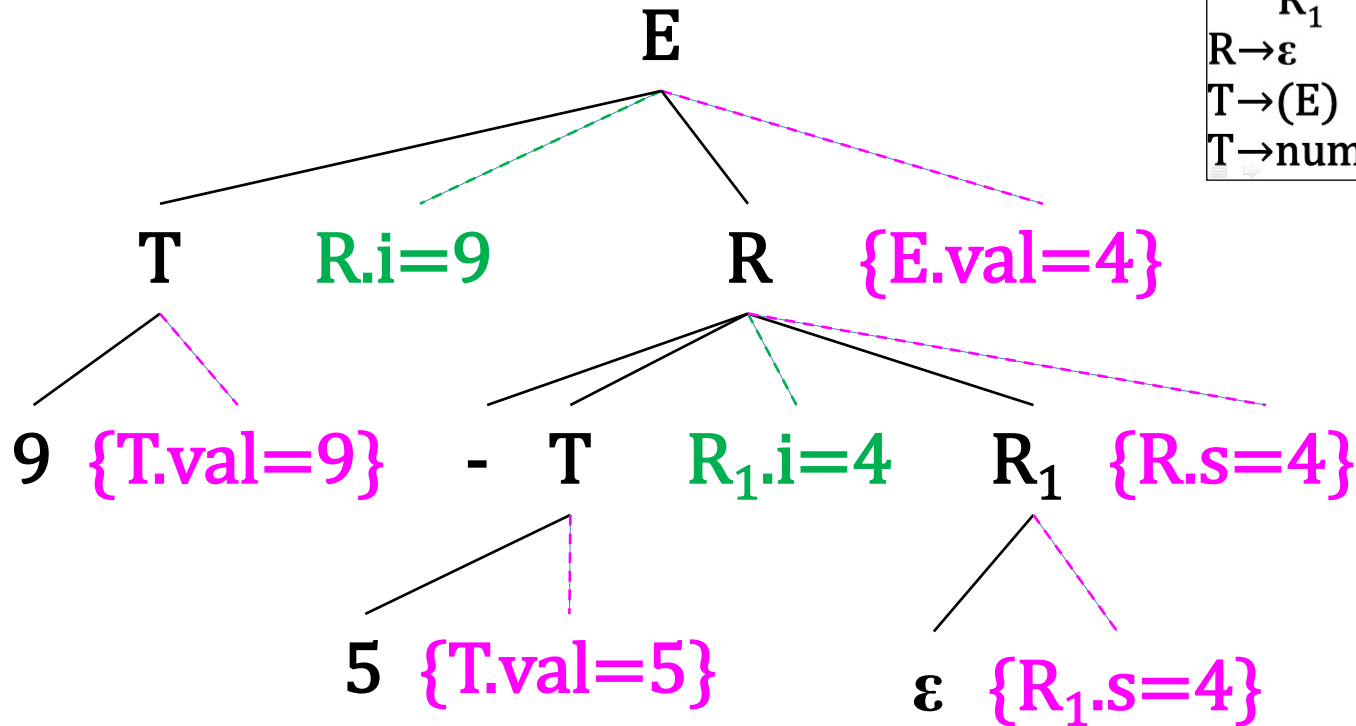
|                             |                            |
|-----------------------------|----------------------------|
| $E \rightarrow T$           | $\{R.i := T.val\}$         |
| $R$                         | $\{E.val := R.s\}$         |
| $R \rightarrow +$           |                            |
| $T$                         | $\{R_1.i := R.i + T.val\}$ |
| $R_1$                       | $\{R.s := R_1.s\}$         |
| $R \rightarrow -$           |                            |
| $T$                         | $\{R_1.i := R.i - T.val\}$ |
| $R_1$                       | $\{R.s := R_1.s\}$         |
| $R \rightarrow \varepsilon$ | $\{R.s := R.i\}$           |
| $T \rightarrow (E)$         | $\{T.val := E.val\}$       |
| $T \rightarrow num$         | $\{T.val := num.val\}$     |

图6.14 消除左递归后的翻译模式



9-5

|                          |                            |
|--------------------------|----------------------------|
| $E \rightarrow T$        | $\{R.i := T.val\}$         |
| $R$                      | $\{E.val := R.s\}$         |
| $R \rightarrow +T$       | $\{R_1.i := R.i + T.val\}$ |
| $R_1$                    | $\{R.s := R_1.s\}$         |
| $R \rightarrow -T$       | $\{R_1.i := R.i - T.val\}$ |
| $R_1$                    | $\{R.s := R_1.s\}$         |
| $R \rightarrow \epsilon$ | $\{R.s := R.i\}$           |
| $T \rightarrow (E)$      | $\{T.val := E.val\}$       |
| $T \rightarrow num$      | $\{T.val := num.val\}$     |



# 构造抽象语法树的翻译模式

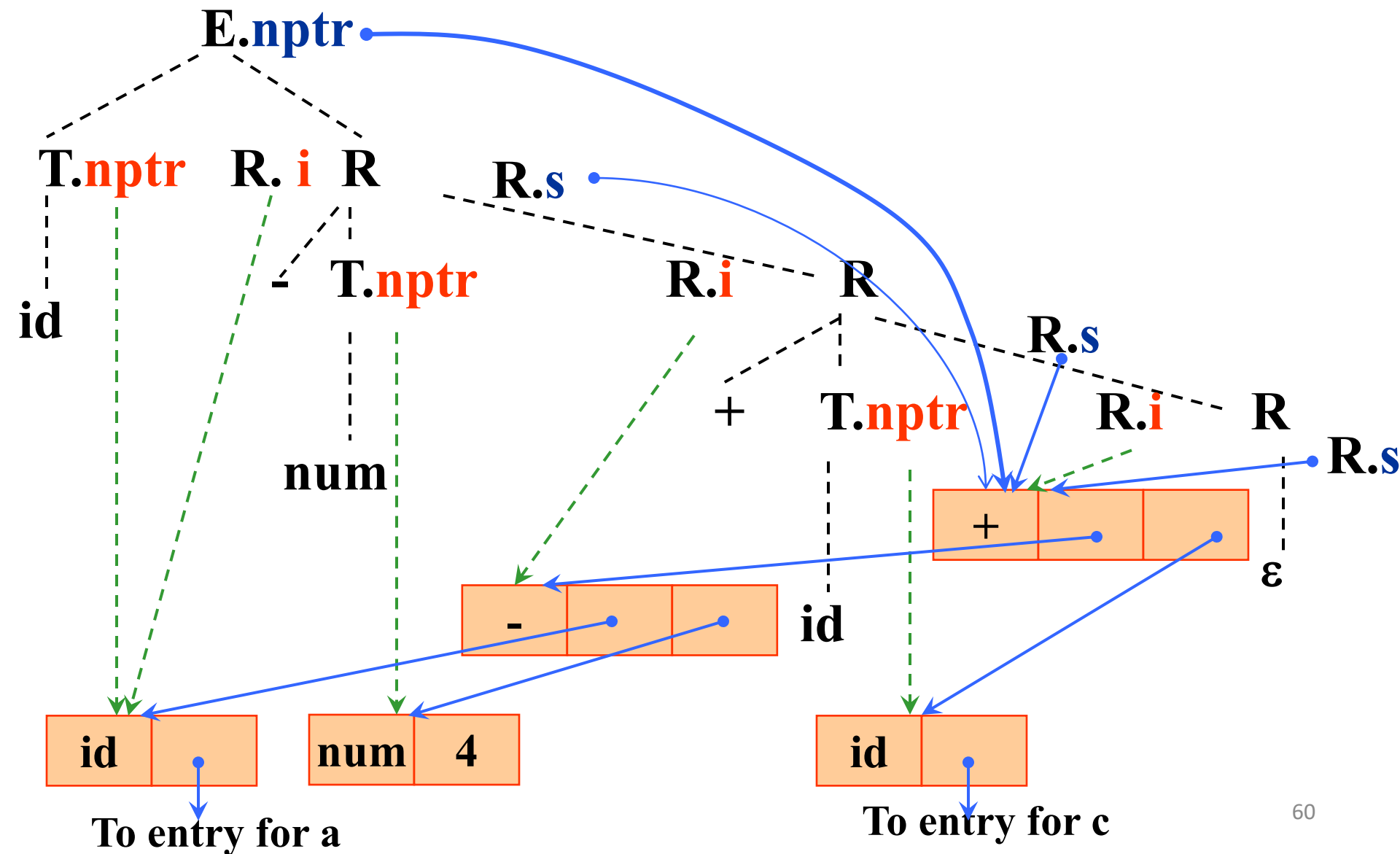
$E \rightarrow E_1 + T \quad \{E.nptr := \text{mknode}('+', E_1.nptr, T.nptr)\}$   
 $E \rightarrow E_1 - T \quad \{E.nptr := \text{mknode}('-', E_1.nptr, T.nptr)\}$   
 $E \rightarrow T \quad \{E.nptr := T.nptr\}$

$T \rightarrow ( E ) \quad \{ T.nptr := E.nptr \}$   
 $T \rightarrow \text{id} \quad \{ T.nptr := \text{mkleaf}(\text{id}, \text{id.entry}) \}$   
 $T \rightarrow \text{num} \quad \{ T.nptr := \text{mkleaf}(\text{num}, \text{num.val}) \}$

# 构造抽象语法树的翻译模式

|                 |               |                                                             |
|-----------------|---------------|-------------------------------------------------------------|
| $E \rightarrow$ | $T$           | $\{ R.i := T.nptr \}$                                       |
|                 | $R$           | $\{ E.nptr := R.s \}$                                       |
| $R \rightarrow$ | $+$           |                                                             |
|                 | $T$           | $\{ R_1.i := \text{mknode}('+', R.i, T.nptr) \}$            |
|                 | $R_1$         | $\{ R.s := R_1.s \}$                                        |
| $R \rightarrow$ | $-$           |                                                             |
|                 | $T$           | $\{ R_1.i := \text{mknode}('-', R.i, T.nptr) \}$            |
|                 | $R_1$         | $\{ R.s := R_1.s \}$                                        |
| $R \rightarrow$ | $\varepsilon$ | $\{ R.s := R.i \}$                                          |
| $T \rightarrow$ | $( E )$       | $\{ T.nptr := E.nptr \}$                                    |
| $T \rightarrow$ | $\text{id}$   | $\{ T.nptr := \text{mkleaf}(\text{id}, \text{id.entry}) \}$ |
| $T \rightarrow$ | $\text{num}$  | $\{ T.nptr := \text{mkleaf}(\text{num}, \text{num.val}) \}$ |

## a-4+c的抽象语法树



## 6.4.3 递归下降翻译器的设计

- 如何在递归下降分析中实现翻译模式，构造递归下降翻译器

## 4.4 递归下降分析程序构造

- 构造不带回溯的自上而下分析器
- 分析程序由一组递归过程组成，文法中每个非终结符对应一个过程；所以这样的分析程序称为递归下降分析器。（因为文法的定义通常是递归的）
- 几个全局过程和变量：
  - ADVANCE，匹配子程序，把输入串指示器IP指向下一个输入符号，即读入一个单词符号
  - SYM，IP当前所指的输入符号
  - ERROR，出错处理子程序

- 例: 文法  $G(E)$  :

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' \mid \varepsilon$$

$$F \rightarrow (E) \mid i$$

- 每个非终结符有对应的子程序的定义，首先在分析过程中，当需要从某个非终结符出发进行展开(推导)时，就调用这个非终结符对应的子程序。

- 例:文法 $G(E)$ :  
     $E \rightarrow TE'$   
     $E' \rightarrow +TE' \mid \varepsilon$   
     $T \rightarrow FT'$   
     $T' \rightarrow *FT' \mid \varepsilon$   
     $F \rightarrow (E) \mid i$
- 对应的递归下降子程序为:

```
void advance(token t)
{
    if(sym==t)

        sym=nextToken;
    else
        error();
}
```



- 例:文法 $G(E)$ :

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid i$

- 对应的递归下降子程序为:

```
void E()
{
    T();
    E'();
}
```

```
void E'()
{
    if(sym=='+')
    {
        advance('+');
        T();
        E'();
    }
}
```

- 例:文法 $G(E)$ :  
     $E \rightarrow TE'$   
     $E' \rightarrow +TE' \mid \varepsilon$   
     $T \rightarrow FT'$   
     $T' \rightarrow *FT' \mid \varepsilon$   
     $F \rightarrow (E) \mid i$
- 对应的递归下降子程序为:

```
void T()  
{  
    F();  
    T'();  
}
```

```
void T'()  
{  
    if(sym=='*')  
    {  
        advance('*');  
        F();  
        T'();  
    }  
}
```

- 例:文法G(E):

$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid \varepsilon$$
$$F \rightarrow (E) \mid i$$

- 对应的递归下降子程序为:

```
void F()
{
    if(sym=='(')
    {
        advance('(');
        E();
        if(sym==')')
            advance('');
        else
            error();
    }
    else if(sym=='i')
        advance('i');
    else
        error();
}
```

- 例:文法 $G(E)$ :  
     $E \rightarrow TE'$   
     $E' \rightarrow +TE' \mid \varepsilon$   
     $T \rightarrow FT'$   
     $T' \rightarrow *FT' \mid \varepsilon$   
     $F \rightarrow (E) \mid i$
- 对应的递归下降子程序为:

```
void main()
{
    sym==nextToken;
    E();
    if(sym!='#')
        error();
}
```

## 6.4.3 递归下降翻译器的设计

### 设计递归下降翻译器的方法

- 1. 对每个非终结符A构造一个函数过程，
  - 对A的每个继承属性设置一个形式参数;
  - 函数的返回值为A的综合属性（作为记录，或指向记录的一个指针，记录中有若干域，每个属性对应一个域）。为了简单，我们假设每个非终结符只有一个综合属性;
  - A对应的函数过程中，为出现在A的产生式中的每一个文法符号的每一个属性都设置一个局部变量。

## 6.4.3 递归下降翻译器的设计

设计递归下降翻译器的方法

- 2. 非终结符A对应的函数过程中，根据当前的输入符号决定使用哪个产生式候选。

## 6.4.3 递归下降翻译器的设计

### 设计递归下降翻译器的方法

- 3. 每个产生式对应的程序代码中，按照从左到右的次序，分别做以下工作：
  - 对带有综合属性 $x$ 的终结符 $X$ 
    - 把属性 $x$ 的值存入为 $X.x$ 设置的变量中;
    - 产生一个匹配 $X$ 的调用;
    - 继续读入一个输入符号。
  - 对非终结符号 $B$ 
    - 产生一个右边带有函数调用的赋值语句  
 $c=B(b_1, b_2, \dots, b_k)$ ,  $b_i (i=1, 2, \dots, k)$ 是为 $B$ 的继承属性设置的变量,  $c$ 是为 $B$ 的综合属性设置的变量。
  - 对每一个语义动作
    - 把动作代码抄进分析器中，用代表属性的变量代替对属性的引用。

## 6.4.3 递归下降翻译器的设计

|                          |                            |                               |
|--------------------------|----------------------------|-------------------------------|
| $E \rightarrow T$        | $\{R.i := T.val\}$         | <code>int E();</code>         |
| $R$                      | $\{E.val := R.s\}$         |                               |
| $R \rightarrow +T$       | $\{R_1.i := R.i + T.val\}$ | <code>int R(int R_in);</code> |
| $R_1$                    | $\{R.s := R_1.s\}$         |                               |
| $R \rightarrow -T$       | $\{R_1.i := R.i -$         |                               |
| $T.val\}$                |                            |                               |
| $R_1$                    | $\{R.s := R_1.s\}$         |                               |
| $R \rightarrow \epsilon$ | $\{R.s := R.i\}$           | <code>int T();</code>         |
| $T \rightarrow (E)$      | $\{T.val := E.val\}$       |                               |
| $T \rightarrow num$      | $\{T.val := num.val\}$     |                               |

图6.14 消除左递归后的翻译模式



|                            |                               |
|----------------------------|-------------------------------|
| $E \rightarrow T$          | $\{R.i := T.val\}$            |
| $R$                        | $\{E.val := R.s\}$            |
| $R \rightarrow +T$         | $\{R_1.i := R.i + T.val\}$    |
| $R_1$                      | $\{R.s := R_1.s\}$            |
| $R \rightarrow -T$         | $\{R_1.i := R.i - T.val\}$    |
| $R_1$                      | $\{R.s := R_1.s\}$            |
| $R \rightarrow \epsilon$   | $\{R.s := R.i\}$              |
| $T \rightarrow (E)$        | $\{T.val := E.val\}$          |
| $T \rightarrow \text{num}$ | $\{T.val := \text{num.val}\}$ |

```
int E(){
    int E_val, R_in, R_s, T_val;
    T_val=T();
    R_in=T_val;
    R_s=R(R_in);
    E_val=R_s;
    return E_val;
}
```

```
int E();
```

```
int R(int R_in);
```

```
int T();
```

```

E→T    {R.i := T.val}
      R    {E.val := R.s}
R→+T    {R1.i := R.i+T.val}
      R1  {R.s := R1.s}
R→-T    {R1.i := R.i-T.val}
      R1  {R.s := R1.s}
R→ε     {R.s := R.i }
T→(E)   {T.val := E.val}
T→num   {T.val := num.val}

```

```
int E();
```

```
int R(int R_in);
```

```
int T();
```

```

int R(int R_in){
    char    lexeme;
    int     R_s, T_val, R1_in, R1_s;
    if( sym=='+' ){
        lexeme=lexval;
        advance( '+' );
        T_val=T();
        R1_in=R_in+T_val;
        R1_s=R(R1_in);
        R_s=R1_s;
    }else if( sym=='-' ){
        lexeme=lexval;
        advance( '-' );
        T_val=T();
        R1_in=R_in-T_val;
        R1_s=R(R1_in);
        R_s=R1_s;
    }else {
        R_s=R_in;
    }
    return R_s;
}

```

```

E→T    {R.i := T.val}
      R    {E.val := R.s}
R→+T    {R1.i := R.i+T.val}
      R1  {R.s := R1.s}
R→-T    {R1.i := R.i-T.val}
      R1  {R.s := R1.s}
R→ε     {R.s := R.i }
T→(E)   {T.val := E.val}
T→num   {T.val := num.val}

```

```
int E();
```

```
int R(int R_in);
```

```
int T();
```

```

int T(){
    char lexeme;
    int T_val, E_val, value;
    if( sym=='(' ){
        lexeme=lexval;
        advance( '(' );
        E_val=E();
        if( sym==')' ){
            lexeme=lexval;
            advance( ')' );
            T_val=E_val;
        }
    }
    else if ( sym=='num' ){
        value=val;
        T_val=value;
    }
    return T_val;
}

```